

Lab 2A working

2a_readme.md

Lab 2 = "Origin Cloaking + CloudFront as the only public ingress."
The clean, realistic interpretation of your requirement (and what big orgs actually do) is:

Only CloudFront is publicly reachable.
ALB is still "internet-facing" (because CloudFront must reach it), but it's cloaked so direct access is blocked:
Security Group allows inbound only from the AWS-managed CloudFront origin-facing prefix list (

2a_lab.txt

Lab 2 Architecture

Internet → CloudFront (+ WAF) → ALB (locked to CloudFront) → Private EC2 → RDS

Key constraints

- No one can hit ALB directly (even if they know ALB DNS)
- WAF enforcement happens at CloudFront edge
- DNS points to CloudFront, not ALB

Terraform Overlay: lab2_cloudfront_origin_cloaking.tf

Assumes you already have:

- aws_lb.chewbacca_alb01
- aws_security_group.chewbacca_alb_sg01
- Route53 zone from Bonus-B
- WAF already exists (you'll replace it with CLOUDFRONT-scoped WAF below)

0) CloudFront must use ACM cert in us-east-1

CloudFront viewer certs must be in N. Virginia (us-east-1). (This is standard AWS behavior; keep it as a lab rule.)

So you will either:

- create a second provider alias for us-east-1, or
- manually create cert and paste ARN (less ideal)

1) Origin cloaking: Allow ALB inbound only from CloudFront prefix list

#lab2_cloudfront_origin_cloaking.tf

Now add/replace your ALB SG ingress:

```
#lab2_cloudfront_origin_cloaking.tf
```

Why this works: AWS provides an AWS-managed prefix list for CloudFront origin-facing IP ranges, maintained by AWS.

2) Add a secret “origin header” that ALB requires
CloudFront will add this header, and ALB will only forward requests that contain it. This is the second layer (defense-in-depth), because anyone can create a CloudFront distribution and use the prefix list trick alone isn’t perfect. AWS explicitly documents the “custom header + ALB rule” approach.

```
#lab2_cloudfront_origin_cloaking.tf
```

Then on the ALB listener, students add a rule:

If header matches → forward to TG

Else → fixed 403

Terraform-wise, this is `aws_lb_listener_rule` on your HTTPS listener.
documents the “custom header + ALB rule” approach.

```
#lab2_cloudfront_origin_cloaking.tf
```

3) WAF moves to CloudFront (WAFv2 scope CLOUDFRONT)

CloudFront uses global scope Web ACLs (scope = "CLOUDFRONT").

```
#lab2_cloudfront_shield_waf.tf
```

4) CloudFront distribution in front of ALB

CloudFront → ALB is a custom origin pattern. You add:

- origin domain = ALB DNS name

- HTTPS only to origin

- custom header X-Chewbacca-Growl with the secret

```
lab2_cloudfront_alb.tf
```

WAF association to CloudFront is done via the distribution config (Terraform `web_acl_id`), consistent with AWS guidance.

5) Route53: point domain to CloudFront (apex + app)

```
lab2_cloudfront_r53.tf
```

#####

What changes from Lab 1C-Bonus-B

✅ Keep

- private EC2, RDS, SSM, Secrets, incident automation, dashboards, alarms

🔄 Modify

WAF moves from ALB → CloudFront
DNS moves from ALB → CloudFront
ALB is “public” technically, but functionally private (only CloudFront can reach it)

- ✓ New control points
 - CloudFront prefix list SG rule
 - ALB secret header rule

Verification CLI (students must prove all 3 requirements)

1) “VPC is only reachable via CloudFront”

A) Direct ALB access should fail (403)

```
curl -I https://<ALB_DNS_NAME>
```

Expected: 403 (blocked by missing header)

B) CloudFront access should succeed

```
curl -I https://chewbacca-growl.com
```

```
curl -I https://app.chewbacca-growl.com
```

Expected: 200/301 → 200

2) WAF moved to CloudFront

```
aws wafv2 get-web-acl \  
--name <project>-cf-waf01 \  
--scope CLOUDFRONT \  
--id <WEB_ACL_ID>
```

And confirm distribution references it:

```
aws cloudfront get-distribution \  
--id <DISTRIBUTION_ID> \  
--query "Distribution.DistributionConfig.WebACLId"
```

Expected: WebACL ARN present.

3) chewbacca-growl.com points to CloudFront

```
dig chewbacca-growl.com A +short
```

```
dig app.chewbacca-growl.com A +short
```

Expected: resolves to CloudFront (you’ll see CloudFront anycast behavior, not ALB IPs)

Lab 2A Summary

Objective: Origin Cloaking + CloudFront as the only public ingress.

Architecture Change:

Internet → CloudFront (+ WAF) → ALB (locked to CloudFront) → Private EC2 → RDS

Key Constraints:

- No one can hit ALB directly (even if they know ALB DNS)
- WAF enforcement happens at CloudFront edge (scope = `CLOUDFRONT`)
- DNS points to CloudFront, not ALB
- ACM cert must be in `us-east-1` for CloudFront viewer cert

Phase 0: Environment Prerequisites (DO THIS FIRST)



CRITICAL: Before running ANY Terraform commands, you MUST verify your AWS region configuration. Mismatched regions will cause phantom errors where resources show the wrong region in `terraform plan` even when your variable is correct.

Step 0.1: Verify AWS CLI Default Region

The AWS provider inherits from your AWS CLI config. This takes precedence over Terraform variables in some cases.

```
# Check current default region
aws configure get region
```

Expected output: `us-east-1`

If it shows `us-east-2` or another region, fix it:

```
# Set the correct default region
aws configure set region us-east-1

# Verify the change
aws configure get region
```

Step 0.2: Check for Environment Variable Overrides

```
# These can override your settings
echo $AWS_DEFAULT_REGION
echo $AWS_REGION

# If either shows wrong region, unset them
```

```
unset AWS_DEFAULT_REGION
unset AWS_REGION
```

Step 0.3: Verify Terraform Variable

In ``03-variables.tf``:

```
variable "aws_region" {
  description = "AWS Region for deployment"
  type        = string
  default     = "us-east-1"
}
```

Step 0.4: Verify Provider Uses the Variable

In ``02-providers.tf``:

```
provider "aws" {
  region =
```

Step 0.5: Clean Slate (If Starting Fresh)

If you have leftover state from previous attempts:

```
# Remove all Terraform state and cache
rm -f terraform.tfstate
rm -f terraform.tfstate.backup
rm -rf .terraform
rm -f .terraform.lock.hcl

# Re-initialize
terraform init
```

Step 0.6: Generate Origin Secret

The `cloudfront_origin_secret` variable has no default (for security). Generate one:

```
openssl rand -base64 32
```

Add to `terraform.tfvars`:

```
cloudfront_origin_secret = "YOUR_GENERATED_SECRET_HERE"
```

Step 0.7: Validate Before Proceeding

```
terraform plan
```

Check the output for:

- All `region =` attributes should show `us-east-1`
- `availability_zone =` should show `us-east-1a`, `us-east-1b`, `us-east-1c`
- No mismatches between region and availability zone

✓ Only proceed to Phase 1 once `terraform plan` shows consistent us-east-1 across all resources.

✓ Only proceed to Phase 1 once `terraform plan` shows consistent us-east-1 across all resources.

Lab 1C Inventory: KEEP vs REMOVE

✓ KEEP (Core Infrastructure from 04-Main.tf)

[illegible]

Resource	Why Remove
<code>aws_wafv2_web_acl.helga_waf01</code>	REMOVE: Scope was REGIONAL, need CLOUDFRONT scope
<code>aws_wafv2_web_acl_association.helga_waf_assoc01</code>	REMOVE: WAF association moves to CloudFront distribution

✗ REMOVE (from 04-1cc-Main.tf - Route53/DNS)

Resource	Why Remove
<code>aws_route53_record.helga_acm_validation_records01</code>	REMOVE: Old cert validation records
<code>aws_acm_certificate_validation.helga_acm_validation01_dns</code>	REMOVE: Old cert validation
<code>aws_route53_record.helga_app_alias01</code>	REMOVE: Will point to CloudFront, not ALB

⚠ KEEP BUT REVIEW (from 04-1cc-Main.tf)

Resource	Notes
<code>aws_route53_zone.helga_zone01</code>	Keep the hosted zone itself
<code>locals.helga_zone_id</code>	Keep the local reference

✗ REMOVE (from 04-1cd-Main.tf)

Resource	Why Remove
<code>aws_route53_record.helga_apex_alias01</code>	REMOVE: Apex will point to CloudFront, not ALB
<code>aws_acm_certificate.helga_cert01</code>	REMOVE: Need us-east-1 cert for CloudFront

⚠ KEEP BUT REVIEW (from 04-1cd-Main.tf)

Resource	Notes
<code>aws_lb.helga_alb01</code>	Keep ALB (but it's now "cloaked")
<code>aws_s3_bucket.helga_alb_logs_bucket01</code>	Keep ALB access logs
<code>aws_s3_bucket_policy.helga_alb_logs_policy01</code>	Keep
<code>aws_route53domains_registered_domain</code>	Keep domain NS sync
<code>`data.aws`_caller_identity.current</code>	Keep
<code>`data.aws`_elb_service_account.main</code>	Keep
<code>`data.aws`_route53_zone.helga_zone</code>	Keep

✗ REMOVE ENTIRELY (04-1ce-Main.tf - WAF Logging)

Resource	Why Remove
ALL WAF logging resources	REMOVE ALL: WAF moves to CloudFront scope. Will recreate logging for CloudFront WAF

Action Plan: Phase 1 (Retraction)

- ☐ **Step 1:** Remove WAF resources (04-1cb) - `helga_waf01` , `helga_waf_assoc01`
- ☐ **Step 2:** Remove old ACM cert resources (04-1cb, 04-1cd) - `helga_acm_cert01` , `helga_cert01` , validation resources
- ☐ **Step 3:** Remove Route53 ALIAS records pointing to ALB (04-1cc, 04-1cd) - `helga_app_alias01` , `helga_apex_alias01`
- ☐ **Step 4:** Remove entire `04-1ce-Main.tf` (WAF logging) - all options
- ☐ **Step 5:** Remove old cert validation records from 04-1cc

Action Plan: Phase 2 (Modification)

- ☐ **Step 6:** Modify ALB SG to use CloudFront prefix list instead of `0.0.0.0/0`
- ☐ **Step 7:** Add secret header listener rule to HTTPS listener

Action Plan: Phase 3 (Addition - Lab 2 New Resources)

- ☐ **Step 8:** Add `us-east-1` provider alias
- ☐ **Step 9:** Create new ACM cert in us-east-1 for CloudFront
- ☐ **Step 10:** Create CloudFront distribution (origin = ALB, secret header, WAF)
- ☐ **Step 11:** Create CloudFront-scoped WAF Web ACL
- ☐ **Step 12:** Create Route53 ALIAS records pointing to CloudFront
- ☐ **Step 13:** Recreate WAF logging for CloudFront WAF

Phase 3 Detailed Instructions

Step 8: Add us-east-1 Provider Alias

File: `02-providers`

CloudFront requires ACM certificates to be in `us-east-1` . Add a provider alias:

```
provider "aws" {
  region = var.aws_region
  # We need an alias for CloudFront (global service)
  alias  = var.aws_region
}
```

Step 9: Create ACM Certificate in us-east-1

File: `04-2a-Main.tf`

This cert covers both apex and app subdomain for CloudFront:

```
resource "aws_acm_certificate" "helga_cf_cert01" {
```

```
provider =
```

Step 10: Create CloudFront Distribution

File: ``04-2ac-Main.tf`` (new file)

 **NEW VARIABLES REQUIRED** (add to ``03-variables.tf``)

```
# =====
# Lab 2A: Origin Cloaking Variables
# =====

variable "cloudfront_origin_secret" {
  description = "Secret header value CloudFront sends to ALB for origin verification. Generate a strong random string."
  type        = string
  sensitive    = true
  # NO DEFAULT - must be provided via terraform.tfvars or TF_VAR_
  # Example: openssl rand -base64 32
}

variable "origin_secret_header_name" {
  description = "HTTP header name for origin cloaking (CloudFront → ALB)"
  type        = string
  default     = "X-Helga-Origin-Verify"
}
```

terraform.tfvars Example

```
# Generate secret: openssl rand -base64 32
cloudfront_origin_secret = "YOUR_GENERATED_SECRET_HERE"
```

CloudFront Distribution Resource

```
#####
# LAB 2A: CLOUDFRONT DISTRIBUTION
#####

# Data source for CloudFront managed prefix list (used in ALB SG)
data "aws_ec2_managed_prefix_list" "cloudfront" {
  name = "
```

How Origin Cloaking Works

1. **CloudFront** sends `X-Helga-Origin-Verify: \\\<secret\\\>` header to ALB
2. **ALB listener rule** checks for this header
3. **If header matches** → forward to target group
4. **If header missing/wrong** → return 403 (direct access blocked)

Step 11: Create CloudFront-Scoped WAF Web ACL

File: `04-2ac-Main.tf`

```
#####
# LAB 2A: CLOUDFRONT-SCOPED WAF WEB ACL
#####

resource "aws_wafv2_web_acl" "helga_cf_waf01" {
  provider      = aws.us_east_1
  name          = "${var.project_name}-cf-waf01"
  description   = "CloudFront WAF for ${var.project_name}"
  scope         = "CLOUDFRONT"

  default_action {
    allow {}
  }

  # Rate limiting rule
  rule {
    name      = "RateLimitRule"
    priority  = 1

    action {
      block {}
    }

    statement {
      rate_based_statement {
        limit              = 2000
        aggregate_key_type = "IP"
      }
    }
  }

  visibility_config {
    cloudwatch_metrics_enabled = true
    metric_name                 = "${var.project_name}-cf-rate-limit"
  }
}
```

```

        sampled_requests_enabled    = true
    }
}

# AWS Managed Rules - Common Rule Set
rule {
    name      = "AWSManagedRulesCommonRuleSet"
    priority = 2

    override_action {
        none {}
    }

    statement {
        managed_rule_group_statement {
            name      = "AWSManagedRulesCommonRuleSet"
            vendor_name = "AWS"
        }
    }

    visibility_config {
        cloudwatch_metrics_enabled = true
        metric_name                 = "${var.project_name}-cf-common-rules"
        sampled_requests_enabled    = true
    }
}

# AWS Managed Rules - Known Bad Inputs
rule {
    name      = "AWSManagedRulesKnownBadInputsRuleSet"
    priority = 3

    override_action {
        none {}
    }

    statement {
        managed_rule_group_statement {
            name      = "AWSManagedRulesKnownBadInputsRuleSet"
            vendor_name = "AWS"
        }
    }

    visibility_config {
        cloudwatch_metrics_enabled = true
    }
}

```

```

        metric_name          = "${var.project_name}-cf-bad-inputs"
        sampled_requests_enabled = true
    }
}

visibility_config {
    cloudwatch_metrics_enabled = true
    metric_name                = "${var.project_name}-cf-waf01"
    sampled_requests_enabled    = true
}

tags = { Name = "${var.project_name}-cf-waf01" }
}

```

Step 12: Create Route53 ALIAS Records → CloudFront

File: `04-2a-Main.tf`

```

#####
# LAB 2A: ROUTE53 ALIAS RECORDS → CLOUDFRONT
#####

# Apex record (williebright.com → CloudFront)
resource "aws_route53_record" "helga_apex_cf_alias01" {
    zone_id = local.helga_zone_id
    name     = var.domain_name
    type     = "A"

    alias {
        name                = aws_cloudfront_distribution.helga_cf_dist01.domain_name
        zone_id             = aws_cloudfront_distribution.helga_cf_dist01.hosted_zone_id
        evaluate_target_health = false
    }
}

# App subdomain record (app.williebright.com → CloudFront)
resource "aws_route53_record" "helga_app_cf_alias01" {
    zone_id = local.helga_zone_id
    name     = local.helga_app_fqdn
    type     = "A"

    alias {
        name                = aws_cloudfront_distribution.helga_cf_dist01.domain

```

```

n_name
    zone_id            = aws_cloudfront_distribution.helga_cf_dist01.hoste
d_zone_id
    evaluate_target_health = false
}
}

```

Step 13: Recreate WAF Logging for CloudFront WAF

File: `04-2ad-Main.tf` (new file)

Choose ONE logging destination (CloudWatch recommended for CloudFront):

Option A: CloudWatch Logs (Recommended)

```

#####
# LAB 2A: WAF LOGGING FOR CLOUDFRONT WAF
#####
# CloudWatch is recommended for CloudFront WAF because:
# - WAF logs must go to us-east-1 for CLOUDFRONT scope
# - CloudWatch in us-east-1 keeps everything in one region
# - Easier to query with Logs Insights

# CloudWatch Log Group (MUST be in us-east-1, MUST start with "aws-waf-logs-")
resource "aws_cloudwatch_log_group" "helga_cf_waf_logs" {
    provider      = aws.us_east_1
    name          = "aws-waf-logs-${var.project_name}-cf"
    retention_in_days = 30

    tags = { Name = "${var.project_name}-cf-waf-logs" }
}

# WAF Logging Configuration
resource "aws_wafv2_web_acl_logging_configuration" "helga_cf_waf_logging" {
    provider          = aws.us_east_1
    log_destination_configs = [aws_cloudwatch_log_group.helga_cf_waf_logs.arn]
    resource_arn      = aws_wafv2_web_acl.helga_cf_waf01.arn
}

```

Variables to Add (`variables.tf`)

```

variable "cloudfront_origin_secret" {
    description = "Secret header value for origin cloaking"
    type        = string
    sensitive   = true
}

```

```

}

variable "origin_secret_header_name" {
  description = "Name of the secret header CloudFront sends to ALB"
  type        = string
  default     = "X-Helga-Origin-Verify"
}

```

ALB Modifications (Phase 2 Reference)

File: ``04-2ab-Main.tf``

These go with Steps 6 & 7:

```

#####
# LAB 2A: ALB SECURITY GROUP (ORIGIN CLOAKING)
#####
# BEFORE: Allowed 0.0.0.0/0 on 80/443
# AFTER: Only allows CloudFront prefix list

# Data source for CloudFront managed prefix list
data "aws_ec2_managed_prefix_list" "cloudfront" {
  name = "com.amazonaws.global.cloudfront.origin-facing"
}

resource "aws_security_group" "helga_alb_sg01" {
  name          = "${var.project_name}-alb-sg01"
  description   = "ALB SG - CloudFront origin-facing only"
  vpc_id        = aws_vpc.helga_vpc01.id

  # HTTPS from CloudFront only
  ingress {
    from_port      = 443
    to_port        = 443
    protocol        = "tcp"
    prefix_list_ids = [data.aws_ec2_managed_prefix_list.cloudfront.id]
    description     = "HTTPS from CloudFront origin-facing IPs only"
  }

  # HTTP from CloudFront only (for redirect)
  ingress {
    from_port      = 80
    to_port         = 80
    protocol        = "tcp"
    prefix_list_ids = [data.aws_ec2_managed_prefix_list.cloudfront.id]
    description     = "HTTP from CloudFront origin-facing IPs only"
  }
}

```

```

    }

    egress {
        from_port    = 0
        to_port      = 0
        protocol     = "-1"
        cidr_blocks  = ["0.0.0.0/0"]
        description  = "Allow all outbound"
    }

    tags = { Name = "${var.project_name}-alb-sg01" }
}

#####
# LAB 2A: HTTPS LISTENER WITH SECRET HEADER RULE
#####

# HTTPS Listener - Default action is 403 (block direct access)
resource "aws_lb_listener" "helga_https_listener01" {
    load_balancer_arn = aws_lb.helga_alb01.arn
    port              = 443
    protocol          = "HTTPS"
    ssl_policy        = "ELBSecurityPolicy-TLS13-1-2-2021-06"
    certificate_arn   = aws_acm_certificate_validation.helga_alb_cert_validation
01.certificate_arn

    # DEFAULT: Block all direct access with 403
    default_action {
        type = "fixed-response"
        fixed_response {
            content_type = "text/plain"
            message_body = "Direct access forbidden"
            status_code  = "403"
        }
    }
}

# Listener Rule: If secret header matches → forward to target group
resource "aws_lb_listener_rule" "helga_origin_verify_rule01" {
    listener_arn = aws_lb_listener.helga_https_listener01.arn
    priority     = 1

    condition {
        http_header {
            http_header_name = var.origin_secret_header_name

```



```

        values          = [var.cloudfront_origin_secret]
    }
}

action {
    type          = "forward"
    target_group_arn = aws_lb_target_group.helga_tg01.arn
}
}

# HTTP Listener - Redirect to HTTPS (unchanged)
resource "aws_lb_listener" "helga_http_listener01" {
    load_balancer_arn = aws_lb.helga_alb01.arn
    port              = 80
    protocol           = "HTTP"

    default_action {
        type = "redirect"
        redirect {
            port          = "443"
            protocol      = "HTTPS"
            status_code    = "HTTP_301"
        }
    }
}

#####
# ALB REGIONAL CERTIFICATE
#####
# This is SEPARATE from the CloudFront cert (us-east-1)
# ALB needs a valid cert for CloudFront → ALB HTTPS connection

resource "aws_acm_certificate" "helga_alb_cert01" {
    domain_name          = var.domain_name
    subject_alternative_names = [local.helga_app_fqdn]
    validation_method     = "DNS"

    tags = { Name = "${var.project_name}-alb-cert01" }

    lifecycle {
        create_before_destroy = true
    }
}

resource "aws_route53_record" "helga_alb_cert_validation" {
    for_each = {

```

```

    for dvo in aws_acm_certificate.helga_alb_cert01.domain_validation_options
: dvo.domain_name => {
    name      = dvo.resource_record_name
    record    = dvo.resource_record_value
    type      = dvo.resource_record_type
  }
}

allow_overwrite = true
name            = each.value.name
records         = [each.value.record]
ttl             = 60
type            = each.value.type
zone_id         = local.helga_zone_id
}

resource "aws_acm_certificate_validation" "helga_alb_cert_validation01" {
  certificate_arn      = aws_acm_certificate.helga_alb_cert01.arn
  validation_record_fqdns = [for record in aws_route53_record.helga_alb_cert_v
validation : record.fqdn]
}

```

⚠ Note: ALB still needs its own regional ACM cert for the CloudFront → ALB HTTPS connection. Keep `helga_alb_cert01` in your regional provider.

Ready to proceed with Phase 1 (Retraction)?

Here's the complete structure for the remaining phases. Copy these sections into your page:

Phase 4: Terraform Deployment Sequence

Since you've done retractions (Phase 1), here's the apply order:

```

terraform init
terraform plan -out=lab2a.plan
terraform apply lab2a.plan

```

Expected Apply Order (Terraform handles dependencies):

1. us-east-1 provider initializes
2. ACM cert in us-east-1 creates → DNS validation records create → validation completes
3. ALB regional cert creates (if new) → DNS validation → validation completes

4. ALB SG updates (CloudFront prefix list)
5. HTTPS listener updates (403 default + header rule)
6. CloudFront WAF creates (us-east-1, CLOUDFRONT scope)
7. CloudFront distribution creates (depends on cert validation + WAF)
8. Route53 ALIAS records update (point to CloudFront)
9. WAF logging configuration creates

 Timing Notes:

- CloudFront distribution creation: **15-30 minutes**
- ACM cert validation: **5-30 minutes** (DNS propagation dependent)
- Total deploy time: **20-45 minutes**

Phase 5: Validation CLI Checklist

✓ 5.1: Verify CloudFront Distribution Deployed

```
# Get distribution ID
aws cloudfront list-distributions \
  --query "DistributionList.Items[?contains(Aliases.Items, '"
```

Expected: Status = Deployed

✓ 5.2: Verify DNS Points to CloudFront

```
dig
```

Expected: Returns CloudFront IPs (not ALB IPs). CloudFront IPs typically resolve to multiple anycast addresses.

```
# Confirm it's CloudFront
dig
```

Expected: Should show `cloudfront.net` in reverse lookup.

✓ 5.3: Verify Origin Cloaking (Direct ALB Access Blocked)

```
# Get ALB DNS name
aws elbv2 describe-load-balancers \
  --names Helga-alb01 \
  --query "LoadBalancers[0].DNSName" \
  --output text
```

```
# Try direct access (should fail with 403)
curl -I https://<ALB_DNS_NAME> --insecure
```

Expected: `HTTP/1.1 403 Forbidden` with body "Direct access forbidden"

✓ 5.4: Verify CloudFront Access Works

```
curl -I
```

Expected: `HTTP/1.1 200 OK` or `301/302` redirect to your app

✓ 5.5: Verify WAF is CloudFront-Scoped

```
# List CloudFront WAFs (must use us-east-1)
aws wafv2 list-web-acls --scope CLOUDFRONT --region us-east-1

# Get specific WAF
aws wafv2 get-web-acl \
  --name Helga-cf-waf01 \
  --scope CLOUDFRONT \
  --id <WEB_ACL_ID> \
  --region us-east-1
```

Expected: WAF exists with scope `CLOUDFRONT`

✓ 5.6: Verify CloudFront Has WAF Attached

```
aws cloudfront get-distribution --id <DISTRIBUTION_ID> \
  --query "Distribution.DistributionConfig.WebACLId"
```

Expected: Returns WAF ARN (not empty/null)

✓ 5.7: Verify WAF Logging Active

```
# Check CloudWatch log group exists (us-east-1)
aws logs describe-log-groups \
  --log-group-name-prefix "aws-waf-logs-Helga" \
  --region us-east-1

# Check WAF logging config
aws wafv2 get-logging-configuration \
  --resource-arn <WAF_ACL_ARN> \
  --region us-east-1
```

Expected: Log group exists and logging config returns successfully

Common Issues & Troubleshooting

● Issue: `terraform init` fails with provider error

Error: `Failed to query available provider packages`

Fix:

```
# Clear provider cache
rm -rf .terraform
rm .terraform.lock.hcl
terraform init -upgrade
```

● Issue: ACM certificate stuck in "Pending validation"

Error: Certificate never validates, CloudFront deploy times out

Troubleshooting:

```
# Check validation records exist
aws acm describe-certificate \
  --certificate-arn <CERT_ARN> \
  --region us-east-1 \
  --query "Certificate.DomainValidationOptions"

# Check DNS records created
dig _
```

Fixes:

1. Ensure Route53 zone ID is correct
2. Check `allow_overwrite = true` on validation records
3. Wait up to 30 min for DNS propagation
4. If hosted zone is new, ensure NS records point to Route53

● Issue: CloudFront creation fails - "CNAMEAlreadyExists"

Error: `One or more of the CNAMEs you provided are already associated with a different resource`

Fix:

1. Check if old CloudFront distribution exists with same aliases
2. Remove aliases from old distribution first
3. Or delete old distribution entirely

```
aws cloudfront list-distributions \
  --query "DistributionList.Items[?contains(Aliases.Items, ']
```

● Issue: CloudFront returns 502 Bad Gateway

Error: CloudFront can't reach ALB origin

Troubleshooting:

```
# Check ALB is healthy
aws elbv2 describe-target-health \
  --target-group-arn <TG_ARN>

# Check ALB SG allows CloudFront
aws ec2 describe-security-groups \
  --group-ids <ALB_SG_ID> \
  --query "SecurityGroups[0].IpPermissions"
```

Fixes:

1. ALB SG must have CloudFront prefix list (not `0.0.0.0/0`)
2. ALB must have valid certificate
3. Target group health checks must pass
4. Origin protocol must be `https-only`

● Issue: CloudFront returns 403 (not from your ALB rule)

Error: WAF is blocking requests

Troubleshooting:

```
# Check WAF logs in CloudWatch
aws logs filter-log-events \
  --log-group-name "aws-waf-logs-Helga-cf" \
  --region us-east-1 \
  --start-time $(date -d '1 hour ago' +%s)000
```

Fix: Review WAF rules, may need to tune rate limits or managed rules

● Issue: "Error: Reference to undeclared resource"

Error: Terraform can't find `aws_vpc.helga_vpc01` or similar

Fixes:

1. Ensure base infrastructure files are still present
2. Check file names match (`.tf` extension)

- **Issue: Provider alias not found**

Fix: Ensure provider block has correct alias:

● Issue: Missing variable `cloudfront_origin_secret`

```
openssl rand -base64 32
```

● Issue: ALB Access Logs - S3 Bucket Permission Denied

```
# Data source to get ELB service account (varies by region)
data "aws_elb_service_account" "main" {}

# S3 bucket for ALB access logs
resource "aws_s3_bucket" "helga_alb_logs_bucket" {
  count = var.enable_alb_access_logs ? 1 : 0
  bucket = "helga-alb-logs-123456789012"
}

# REQUIRED: Bucket policy allowing ELB to write logs
resource "aws_s3_bucket_policy" "helga_alb_logs_policy" {
  count = var.enable_alb_access_logs ? 1 : 0
}
```

```
bucket = aws_s3_bucket.helga_alb_logs_bucket[0].id

policy = jsonencode({
  Version = "2012-10-17"
  Statement = [
    {
      Effect = "Allow"
      Principal = {
        AWS =
```

Quick CLI Fix (if you need to test immediately):

```
# ELB service account for us-east-1 is 127311923021
aws s3api put-bucket-policy --bucket helga-alb-logsv1-919113286081 --policy '{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::127311923021:root"
      },
      "Action": "s3:PutObject",
      "Resource": "arn:aws:s3:::helga-alb-logsv1-919113286081/AWSLogs/91911328
6081/*"
    }
  ]
}'
```

Key Point: The bucket policy MUST exist BEFORE the ALB tries to enable access logging. Terraform handles this dependency automatically if both resources are in the same config.

Issue: WAF logging fails - invalid log destination

Error: `WAFLogDestinationPermissionIssue`

Fixes:

1. Log group name MUST start with `aws-waf-logs-`
2. Log group MUST be in `us-east-1` for CloudFront WAF
3. Ensure provider on log group is `'aws.us'_east_1`

Success Criteria Checklist

- ☐ `terraform apply` completes without errors
- ☐ CloudFront distribution status = `Deployed`

- ☐ `dig williebright.com` returns CloudFront IPs
- ☐ Direct ALB access returns `403 Forbidden`
- ☐ CloudFront access returns `200 OK`
- ☐ WAF is CLOUDFRONT-scoped and attached to distribution
- ☐ WAF logs flowing to CloudWatch

Copy all of that into your page. Run `terraform init` and let me know what errors you hit—I'll help you troubleshoot.

You're right—Terraform is trying to create resources that already exist in AWS. Here's how to import them:

Import Commands

Run these one at a time (replace region if needed):

```
# 1. Secrets Manager Secret
terraform import 'aws_secretsmanager_secret.helga_db_secretlab2a' 'helga/rds/mysqlv2'

# 2. CloudWatch Log Group
terraform import 'aws_cloudwatch_log_group.helga_log_grouplab2a' '/aws/ec2/helga-rds-app'

# 3. S3 Bucket (note the [0] index)
terraform import 'aws_s3_bucket.helga_alb_logs_bucketlab2a[0]' 'helga-alb-logs-919113286081'
```

VPC Limit Issue (Different Problem)

The VPC error is **not** an import issue—you've hit AWS's VPC limit (default is 5 per region). You have two options:

Option A: Delete an old/unused VPC first

```
# List your VPCs to see what exists
aws ec2 describe-vpcs --query "Vpcs[*].{ID:VpcId,CIDR:CidrBlock,Name:Tags[?Key=='Name']|[0].Value}" --output table
```

Then delete the one you don't need (make sure it's not Lab 1C's if you still want that).

Option B: Import the existing VPC if it's the one you want to use

```
# Get the VPC ID first
aws ec2 describe-vpcs --filters "Name=tag:Name,Values=*helga*" --query "Vpcs[*].VpcId" --output text
```

```
# Then import it
terraform import 'aws_vpc.helga_vpclab2a' 'vpc-XXXXXXXX'
```

SNS Topic

That SNS error is strange—it says the topic doesn't exist but also fails to create it. After you resolve the VPC issue and run the imports above, try `terraform plan` again. If the SNS error persists, you may need to manually create or import it:

```
# Check if it exists
aws sns list-topics --query "Topics[?contains(TopicArn, 'helga')]"

# If it exists, import it
terraform import 'aws_sns_topic.helga_sns_topic' 'arn:aws:sns:us-east-2:123456789012:helga-db-incidents'
```

After Imports

Once all imports complete, run:

```
terraform plan
```

This should show a much cleaner diff since Terraform now knows about the existing resources.

Setting a clean slate for files. removing prior dependencies.

Yes, for a clean fresh start, delete all state files:

```
# Delete state files
rm terraform.tfstate
rm terraform.tfstate.backup

# Also clear the .terraform directory (provider cache + modules)
rm -rf .terraform
rm .terraform.lock.hcl
```

Then Re-Initialize

```
terraform init
terraform plan
```

Why This Works

- `terraform.tfstate` — Terraform's record of what it thinks exists in AWS
- `terraform.tfstate.backup` — Previous version of the state file
- `.terraform/` — Cached providers and modules

If you've already destroyed everything in AWS and want to start fresh, these files are just holding references to resources that no longer exist. Deleting them gives you a blank slate.

After `terraform init` and `terraform plan`, you should see Terraform planning to **create** all resources fresh in the correct region (us-east-1), with no leftover us-east-2 references.

That bucket **already exists in us-east-2** from a previous deployment. S3 bucket names are globally unique across all regions—you can't create a bucket with the same name in a different region.

Your Options

Option 1: Delete the existing bucket in us-east-2 (Recommended for clean slate)

```
# First, empty the bucket (required before deletion)
aws s3 rm s3://helga-alb-logs-919113286081 --recursive --region us-east-2

# Then delete the bucket
aws s3api delete-bucket --bucket helga-alb-logs-919113286081 --region us-east-2
```

Then run `terraform apply` again—it will create fresh in us-east-1.

Option 2: Use a different bucket name

In your Terraform code, change the bucket name to include the region or a unique suffix:

```
bucket = "helga-alb-logs-${
```

Recommendation

Go with **Option 1**—delete the old bucket and let Terraform create it fresh in us-east-1. This keeps everything in one region and maintains consistency.

Phase 6: Post-Deploy Validation & Verification



Run these steps **AFTER** `terraform apply` completes successfully to verify Lab 2A is working correctly.

6.1: Get Resource Identifiers

First, gather the IDs you'll need for verification:

```
# Get CloudFront Distribution ID
CF_DIST_ID=$(aws cloudfront list-distributions --query "DistributionList.Items
[?contains(Aliases.Items, 'williebright.com')].Id" --output text)
echo "CloudFront Distribution ID: $CF_DIST_ID"

# Get WAF Web ACL ID
WAF_ID=$(aws wafv2 list-web-acls --scope CLOUDFRONT --region us-east-1 --query
"WebACLS[?contains(Name, 'helga')].Id" --output text)
echo "WAF ID: $WAF_ID"

# Get ALB ARN
ALB_ARN=$(aws elbv2 describe-load-balancers --query "LoadBalancers[?contains(L
oadBalancerName, 'helga')].LoadBalancerArn" --output text)
echo "ALB ARN: $ALB_ARN"

# Get Target Group ARN
TG_ARN=$(aws elbv2 describe-target-groups --query "TargetGroups[?contains(Targ
etGroupName, 'helga')].TargetGroupArn" --output text)
echo "TG ARN: $TG_ARN"

# Get ALB DNS Name
ALB_DNS=$(aws elbv2 describe-load-balancers --query "LoadBalancers[?contains(L
oadBalancerName, 'helga')].DNSName" --output text)
echo "ALB DNS: $ALB_DNS"
```

```
# Resource Identifiers (Sanitized)
WAF ID: xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx
ALB ARN: arn:aws:elasticloadbalancing:us-east-1:123456789012:loadbalancer/app/helga-alb/XXXXXXXXXX
TG ARN: arn:aws:elasticloadbalancing:us-east-1:123456789012:targetgroup/helga-tg/XXXXXXXXXX
ALB DNS: helga-alb-XXXXXXXXXX.us-east-1.elb.amazonaws.com
```

6 Old test from

Got it. Here's the **exact lab verification CLI** formatted for Git Bash (your domain is ``williebright.com``, project is `helga`):

```
# =====
# VERIFICATION CLI - Lab 2A (3 Requirements)
# =====

# -----
```

```
# 1) VPC is only reachable via CloudFront
```

```
# -----
```

```
# A) Direct ALB access should fail (403)
```

```
curl -I https://<ALB_DNS_NAME>
```

```
curl -I https://
```

	XXXXXXXXXXXXXXXX	
aws wafv2 get-web-acl --name helga-cf-waf01 --scope CLOUDFRONT --id t-xxxxxxx --region us-east-1	t-xxxxxxx	

```
arn:aws:wafv2:us-east-1:123456789012:global/webacl/CreatedByCloudFront-XXXXXXX/XX
```

```
aws wafv2 get-web-acl --name helga-cf-waf --scope CLOUDFRONT --id <WAF_ID> --region us-east-1
```

Replace `<ALB_DNS_NAME>`, `<WEB_ACL_ID>`, and `<DISTRIBUTION_ID>` with your actual values from your deployment.

```
brigh@DESKTOP-62DEPP9 MINGW64 ~/Documents/TheoWAF/class7/AWS/Classes/armageddon_4_Lab_2a (master)
$ curl -i https://helga-alblab2a-505583442.us-east-1.elb.amazonaws.com
curl: (28) Failed to connect to helga-alblab2a-505583442.us-east-1.elb.amazonaws.com port 443 after 21366 ms
: Could not connect to server
```

```
brigh@DESKTOP-62DEPP9 MINGW64 ~/Documents/TheoWAF/class7/AWS/Classes/armageddon_4_Lab_2a (master)
$ curl -I https://williebright.com
HTTP/1.1 404 Not Found
Content-Type: text/html; charset=utf-8
Content-Length: 207
Connection: keep-alive
Date: Sat, 17 Jan 2026 22:12:36 GMT
Server: Werkzeug/3.1.5 Python/3.9.25
X-Cache: Error from cloudfront
Via: 1.1 e5390f800690cfa4bd359d02b9b69064.cloudfront.net (CloudFront)
X-Amz-Cf-Pop: ORD58-P5
X-Amz-Cf-Id: __8jCXSuKepZAbNXHA38Iq1cusISFF4Hjbo1vXi5DJHsHBBZUnuZSw==
```

```
brigh@DESKTOP-62DEPP9 MINGW64 ~/Documents/TheoWAF/class7/AWS/Classes/armageddon_4_Lab_2a (master)
$ curl -I https://app.williebright.com
HTTP/1.1 404 Not Found
Content-Type: text/html; charset=utf-8
Content-Length: 207
Connection: keep-alive
Date: Sat, 17 Jan 2026 22:53:15 GMT
Server: Werkzeug/3.1.5 Python/3.9.25
X-Cache: Error from cloudfront
Via: 1.1 fbba163d825ae4c940f6fb8e6dd730bc.cloudfront.net (CloudFront)
X-Amz-Cf-Pop: ORD58-P5
X-Amz-Cf-Id: _DHeNltggWFDjUbt71VtD5505VBaRuj4VEI9segLKSdV_QH0f_6DnQ==
```

```

brigh@DESKTOP-62DEPP9 MINGW64 ~/Documents/TheoWAF/class7/AWS/Classes/armageddon_4_Lab_2a (master)
$ aws wafv2 get-web-acl --name helga-cf-waflab2a --scope CLOUDFRONT --id 9c64a0cf-6d67-4bbe-82b7-c61c05b2d726 --region us-east-1
{
  "WebACL": {
    "Name": "helga-cf-waflab2a",
    "Id": "9c64a0cf-6d67-4bbe-82b7-c61c05b2d726",
    "ARN": "arn:aws:wafv2:us-east-1:919113286081:global/webacl/helga-cf-waflab2a/9c64a0cf-6d67-4bbe-82b7-c61c05b2d726",
    "DefaultAction": {
      "Allow": {}
    },
    "Description": "CloudFront WAF for helga",
    "Rules": [
      {
        "Name": "RateLimitRule",
        "Priority": 1,
        "Statement": {
          "RateBasedStatement": {
            "Limit": 2000,
            "EvaluationWindowSec": 300,
            "AggregateKeyType": "IP"
          }
        },
        "Action": {
          "Block": {}
        },
        "VisibilityConfig": {
          "SampledRequestsEnabled": true,

```

```

          "RateBasedStatement": {
            "Limit": 2000,
            "EvaluationWindowSec": 300,
            "AggregateKeyType": "IP"
          }
        },
        "Action": {
          "Block": {}
        },
        "VisibilityConfig": {
          "SampledRequestsEnabled": true,
          "CloudWatchMetricsEnabled": true,
          "MetricName": "helga-cf-rate-limit"
        }
      },
      {
        "Name": "AWSManagedRulesCommonRuleSet",
        "Priority": 2,
        "Statement": {
          "ManagedRuleGroupStatement": {
            "VendorName": "AWS",
            "Name": "AWSManagedRulesCommonRuleSet"
          }
        },
        "OverrideAction": {
          "None": {}
        },
        "VisibilityConfig": {

```

```
brigh@DESKTOP-62DEPP9 MINGW64 ~/Documents/TheoWAF/class7/AWS/Classes/armageddon_4_Lab_2a (master)
$ aws wafv2 get-web-acl --name helga-cf-waflab2a --scope CLOUDFRONT --id 9c64a0cf-6d67-4bbe-82b7-c61c05b2d726 --region us-east-1
{
  "OverrideAction": {
    "None": {}
  },
  "VisibilityConfig": {
    "SampledRequestsEnabled": true,
    "CloudWatchMetricsEnabled": true,
    "MetricName": "helga-cf-bad-inputs"
  }
},
{
  "VisibilityConfig": {
    "SampledRequestsEnabled": true,
    "CloudWatchMetricsEnabled": true,
    "MetricName": "helga-cf-waflab2a"
  },
  "Capacity": 902,
  "ManagedByFirewallManager": false,
  "LabelNamespace": "aws:wafv2:us-east-1:webacl:helga-cf-waflab2a:",
  "RetrofittedByFirewallManager": false,
  "OnSourceDDoSProtectionConfig": {
    "ALBLowReputationMode": "ACTIVE_UNDER_DDOS"
  }
},
{
  "LockToken": "486419ac-c0bf-4838-89c0-48ea1c8c1244"
}
}
```

```
brigh@DESKTOP-62DEPP9 MINGW64 ~/Documents/TheoWAF/class7/AWS/Classes/armageddon_4_Lab_2a (master)
● $ aws cloudfront get-distribution --id E1TVVCTL7Q4QCW --query "Distribution.DistributionConfig.WebACLId"
"arn:aws:wafv2:us-east-1:919113286081:global/webacl/helga-cf-waflab2a/9c64a0cf-6d67-4bbe-82b7-c61c05b2d726"
```

```
brigh@DESKTOP-62DEPP9 MINGW64 ~/Documents/TheoWAF/class7/AWS/Classes/armageddon_4_Lab_2a (master)
● $ nslookup williebright.com
Server:   arrisatom.bloombb.com
Address:  192.168.0.1

Non-authoritative answer:
Name:     williebright.com
Addresses: 3.23.25.171
          18.216.38.254

brigh@DESKTOP-62DEPP9 MINGW64 ~/Documents/TheoWAF/class7/AWS/Classes/armageddon_4_Lab_2a (master)
● $ nslookup app.williebright.com
Server:   arrisatom.bloombb.com
Address:  192.168.0.1

Non-authoritative answer:
Name:     app.williebright.com
Addresses: 18.160.249.45
          18.160.249.55
          18.160.249.116
          18.160.249.128
```

```
brigh@DESKTOP-62DEPP9 MINGW64 ~/Documents/TheoWAF/class7/AWS/Classes/armageddon_4_Lab_2a (master)
$ aws route53 list-resource-record-sets --hosted-zone-id Z076499737UY78K9J1ZVE --output text
RESOURCERECORDSETS    williebright.com.      A
ALIASTARGET            d2wmipbg1k98vc.cloudfront.net.  False  Z2FDTNDATAQYW2
RESOURCERECORDSETS    williebright.com.      172800  NS
RESOURCERECORDS        ns-1838.awsdns-37.co.uk.
RESOURCERECORDS        ns-5.awsdns-00.com.
RESOURCERECORDS        ns-906.awsdns-49.net.
RESOURCERECORDS        ns-1480.awsdns-57.org.
RESOURCERECORDSETS    williebright.com.      900     SOA
RESOURCERECORDS        ns-1838.awsdns-37.co.uk.  awsdns-hostmaster.amazon.com. 1 7200 900 1209600 86400
RESOURCERECORDSETS    _9dcb7487f1570f2a1f57f926614a020d.williebright.com.  60     CNAME
RESOURCERECORDS        _03d5e8cbb6ddfdee5ecc6aca5284491.jkddzztszm.acm-validations.aws.
RESOURCERECORDSETS    app.williebright.com.  A
ALIASTARGET            d2wmipbg1k98vc.cloudfront.net.  False  Z2FDTNDATAQYW2
```

✓ Lab 2A Verification: COMPLETE



Lab 2A Origin Cloaking successfully deployed and verified on January 17, 2026.

Verification Results

Requirement	Status	Evidence
1) VPC only reachable via CloudFront	✓ PASSED	Direct ALB access times out (blocked by SG prefix list)
2) WAF moved to CloudFront	✓ PASSED	<code>helga-cf-waflab2a</code> with scope <code>CLOUDFRONT</code> , attached to distribution
3) Domain points to CloudFront	✓ PASSED	Both <code>example.com</code> and <code>app.example.com</code> ALIAS to CloudFront distribution

Key Resources Deployed

- CloudFront Distribution: `XXXXXXXXXXXXXX`
- WAF Web ACL ID: `XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX`
- ALB: `helga-alb-XXXXXXXX.us-east-1.elb.amazonaws.com` (cloaked — CloudFront only)
- Target Group: `helga-tg`
- Route53 Hosted Zone: `ZXXXXXXXXXXXXX`

Architecture Achieved

Internet → CloudFront (+ WAF) → ALB (locked to CloudFront) → Private EC2 → RDS